Artificial Intelligence

Mini Project 01

Escape the Maze

Search Algorithms and Heuristic Evaluation

Project Report

GitHub Repository: reyanmuzamil/maze-runner

Reyan Muzammil BSCS25009

Tayyab Ahmed Minhs BSCS25031

October 2026

Table of Contents

1 **AI Assistant Used** 1

2 **Experimental Results** 1

3 **Depth-First Search versus Breadth-First Search** 2

4 **Iterative Deepening Search** 2

5 **Path Costs: BFS versus Uniform-Cost Search** 3

6 **Heuristics: Greedy Search and A*** 3

7 **Q9: Key-Hunt Heuristic** 4

Report Overview

This report investigates six classical search algorithms — DFS, BFS, IDS, UCS, Greedy Best-First, and A* — applied to maze navigation tasks of varying complexity. We document the AI assistance used, present empirical performance results across three maze environments, and analyse the trade-offs between solution optimality, completeness, memory usage, and running time. A custom consistent heuristic for the key-hunt variant is derived and evaluated against the assignment's expansion thresholds.

1. AI Assistant Used

We used **ChatGPT** to help us understand search concepts, discuss debugging issues, interpret our experimental results. The maze algorithms were tested through our own local runs of the project. The numerical results reported below are taken from those terminal outputs, rather than generated or estimated by the AI assistant.

2. Experimental Results

We used the project repository's `-compare` option on the three mazes required by the assignment (Table 1, Table 2, and Table 3). *Path cost* is the sum of terrain-entry costs, *steps* is the number of moves, and *expanded* counts calls that expand search states. A timeout means no completed result was returned within the comparison's ten-second limit. Times are measured in seconds.

Table 1. Medium maze (21 × 41)

Algorithm	Cost	Steps	Expanded	Time (s)
DFS	162	162	233	0.002
BFS	66	66	407	0.001
IDS	66	66	437,730	1.339
UCS	66	66	407	0.001
Greedy (Manhattan)	80	80	81	0.000
A* (Manhattan)	66	66	209	0.001

Table 2. Open Glade (15 × 33)

Algorithm	Cost	Steps	Expanded	Time (s)
DFS	234	234	322	0.002
BFS	32	32	344	0.001
IDS	—	—	2,722,500	10.000
UCS	32	32	344	0.001
Greedy (Manhattan)	36	36	39	0.000
A* (Manhattan)	32	32	114	0.000

IDS timed out after 10 seconds.

Table 3. Ivy Maze (21 × 41)

Algorithm	Cost	Steps	Expanded	Time (s)
DFS	159	88	422	0.003
BFS	80	60	439	0.001
IDS	—	—	3,253,968	10.000
UCS	70	64	418	0.001
Greedy (Manhattan)	130	82	85	0.000
A* (Manhattan)	70	64	344	0.001

IDS timed out after 10 seconds.

3. Depth-First Search versus Breadth-First Search

On `maze_medium`, DFS takes **162 steps**, while BFS takes only **66**. DFS follows one branch as far as possible before backtracking. The first exit it discovers can therefore lie at the end of a long detour. BFS explores states in increasing order of depth, so it finds a route with the fewest actions.

For a finite maze using a graph-search explored set, **both DFS and BFS are complete**: if a reachable exit exists, they will eventually find it. DFS is **not optimal**, because its first solution may be unnecessarily long. BFS is **optimal in number of steps** when every action has equal cost; it is also optimal in total path cost when all actions have the same positive cost. When terrain costs differ, BFS can find a shorter route that costs more.

4. Iterative Deepening Search

IDS runs depth-limited DFS repeatedly with depth limits of $0, 1, 2, \ldots$ until it finds the exit. Our DLS avoids cycles by checking states on the *current path*, not by permanently excluding states visited in other branches. This allows the same position to be explored again through many different routes.

On `maze_small`, the corridors have no loops, limiting the number of alternative routes and making IDS manageable. On the open `open_glade` maze, many distinct simple paths reach similar positions. The path-only cycle check prevents immediate loops but does not prevent this repeated work across branches and depth limits. As a result, IDS expanded **2,722,500 nodes** before the ten-second timeout, even though BFS found the exit after **344 expansions**. IDS also timed out on `ivy_maze`.

The benefit IDS offers over BFS is **lower memory use**: depth-first exploration mainly stores the current search path, while BFS must keep a broad information on to be explored states in memory. When step costs are equal and the search finishes, IDS, like BFS, finds a shallowest solution.

5. Path Costs: BFS versus Uniform-Cost Search

BFS minimizes the number of actions. UCS minimizes the accumulated movement cost $g(n)$, accounting for expensive terrain. On `griever_alley`, our tests produced:

Algorithm	Path cost	Steps	Expanded
BFS	56	20	42
UCS	42	28	50

BFS chooses the shorter route, even though it crosses costly Griever territory. UCS accepts eight extra steps to avoid enough expensive cells to reduce the path cost from **56 to 42**. They are guaranteed to return equally good minimum-cost paths when every legal move has the same positive cost. More generally, they may happen to agree on a particular maze, but equal costs are the straightforward guarantee.

6. Heuristics: Greedy Search and A*

Greedy best-first search chooses the state with the smallest heuristic estimate $h(n)$ alone. On `greedy_trap`, the exit appears close in straight-line grid distance, but a wall and costly terrain make that direction misleading. Greedy follows this attractive-looking route and returns a path costing **52**, expanding **60 nodes**. A* also considers the cost already paid, using

$$f(n) = g(n) + h(n), \tag{1}$$

so it finds a route costing **25**, though it expands more nodes (**155**). A low expansion count does not necessarily mean a better path.

The Euclidean heuristic is **admissible** here: straight-line distance never exceeds the distance along a route made of horizontal and vertical moves, and each move costs at least one. Manhattan distance is also admissible and is generally a stronger lower bound for four-direction movement.

Our A* results for `maze_big` are:

Heuristic	Path cost	Steps	Expanded
Manhattan	124	124	679
Euclidean	124	124	880

Both heuristics preserve the cheapest path cost of **124**. Manhattan expands **201 fewer nodes**, because its estimate is at least as large as Euclidean distance for grid positions, while still not overestimating the remaining cost.

7. Q9: Key-Hunt Heuristic

In the key-hunt problem, Thomas must collect *every* remaining code piece before reaching the exit. The search state records his current position p and the remaining key positions K. Our heuristic uses the provided `mazeDistance` function, which gives the shortest number of steps between maze positions and caches distances.

When keys remain, we estimate the distance of a route that must pass through each individual remaining key k and then reach the exit E, taking the maximum:

$$h(p, K) = \max_{k \in K} \big[d(p, k) + d(k, E) \big]. \tag{2}$$

When no keys remain, we use $h(p, \varnothing) = d(p, E)$, which is zero when Thomas reaches the exit.

Why it is consistent. Our heuristic is consistent because one move cannot reduce the estimated distance by more than the cost of that move. If Thomas moves without collecting a key, the distance to each remaining key may change, but it cannot decrease by more than the move costs. The distance from that key to the exit stays the same, so the estimate follows the consistency rule:

$$h(s) \leq c + h(s'). \tag{3}$$

Here, s is Thomas's state before moving, s' is his state after moving, c is the cost of that move, and h is the estimated travel cost still remaining. In simple terms, the estimate before a move must not be greater than the move's cost plus the estimate afterward. For example, if a move costs 1 and the new estimate is 9, the old estimate must be at most 10. An estimate that drops from 20 to 9 after a cost-1 move would break this rule.

If Thomas collects a key, we remove it from the remaining keys. However, he still needs to reach the exit. The new estimate is at least the shortest distance from his position to the exit, whether or not other keys remain. Since he has just stepped onto the collected key, the old estimate for that key cannot be greater than the move cost plus the new estimate. The same rule also holds

for all other keys. Taking the maximum of these estimates keeps the heuristic consistent. Once Thomas has collected all the keys and reached the exit, the estimate is zero because no travel remains.

Test Results. We tested A* using our key-hunt heuristic on the `keys_medium` maze. The results are shown in Table 1.

Table 1: A* Key-Hunt Heuristic Results

Metric	**Result**
Maze	`keys_medium`
Algorithm	A* Search
Heuristic	Key Hunt
Path Cost	94
Steps	94
Nodes Expanded	1,678
Search Time	0.083 s